RASPBERry: Enabling Continuous DRL at the Edge

Chengming Zhou
School of Computing
Newcastle University
Newcastle upon Tyne, UK
c.zhou10@newcastle.ac.uk

Stephen Mcgough*
School of Computing
Newcastle University
Newcastle upon Tyne, UK
stephen.mcgough@newcastle.ac.uk

Jacek Cala
School of Computing
Newcastle University
Newcastle upon Tyne, UK
jacek.cala@newcastle.ac.uk

Abstract

Replay buffers in Deep Reinforcement Learning (DRL) often demand extensive computation and memory, especially as buffer capacity grow. We propose **RASPBERry** (RAM-Saving Prioritized Block Experience Replay), a novel method that reorganizes experience buffers into compressed blocks. By processing far fewer elements overall and reducing overhead in *Sample* and *Update*, RASPBERry significantly cuts down both computational cost and DRAM usage—all without sacrificing training effectiveness.

We demonstrate RASPBERry on top of standard Prioritized Experience Replay (PER) and its distributed implementation (DPER), and evaluate it across a diverse set of environments, including Atari, Box2D, and MiniGrid. Empirical results consistently show faster learning curves and up to 95% lower memory requirements compared to the original PER. On a normal laptop, RASPBERry achieves comparable performance to large scale Ape-X DDQN.

Crucially, our method also enables DRL training on memory-limited devices: with only **8 GB** of DRAM, we can run **one million** Atari transitions and solve Pong on an NVIDIA Jetson TX2-like setup by DDQN, a task infeasible for traditional PER. Moreover, experiments on HPC cluster confirm that RASPBERry's benefits scale to larger workloads and buffer sizes. These findings suggest RASPBERry as a practical drop-in replacement for PER, bringing high-throughput DRL to both edge devices and large-scale computing platforms.

Keywords: Deep Reinforcement Learning, Deep Q-Learning,
Distributed Deep Reinforcement Learning, Deep Q-Learning
Experience Replay, Memory-Constrained Hardware

Acknowledgements

We are deeply indebted to the JADE HPC facility in the UK for generously providing the computational resources that made our large-scale experiments possible. We would also like to thank Newcastle University for its continuous support.

*Corresponding author. Email: stephen.mcgough@newcastle.ac.uk

1 Introduction

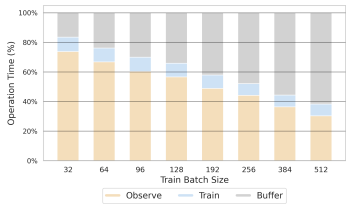
Deep Reinforcement Learning (DRL)[9] has achieved remarkable success by combining Reinforcement Learning (RL) with Deep Learning (DL). A core mechanism that stabilises DRL training is *Experience Replay* (ER) [8], which stores recent transitions in a replay buffer for repeated sampling. Prioritised Experience Replay (PER) further improves ER by sampling based on the Temporal Difference (TD) error—often referred to as a **surprise** signal [1, 14]—and has been demonstrated to boost sample efficiency [5]. However, PER can be computationally expensive, especially with large replay buffers and mini-batch sizes, and maintaining an ER often implies high memory usage [7].

Existing solutions aim to address part of this problem. For instance, implementations like Rainbow [11] and Stable-Baselines 3 [12] partially alleviate overhead by using fixed-size tensor arrays. Yet, this approach is less compatible with dynamic replay buffers [15] and large-scale distributed frameworks [10]. When replay buffers are distributed across many actors in the Ape-X framework [6], such as Ray rllib [10]), it could solved by larger training intensity, smaller mini-batch size with lots of asynchronous Replay Buffer Actor. However,both computation requirements and memory usage would be quite high.

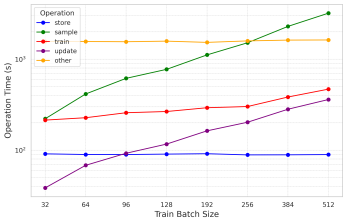
Building on these advances and inspirations from other contributions, we propose **RASPBERry** to reduce DRAM cost. Rather than storing or updating each transition individually, RASPBERry groups consecutive transitions into blocks, assigns a single priority value to each block, and compress it. This significantly lowers the processing complexity and memory footprint, making it particularly suitable for resource-constrained hardware such as raspberry Pi, Nvidia Jetson Nano or Laptops. Our experiments show that **RASPBERry** achieves comparable or better performance than existing experience replay solutions while dramatically reducing memory usage (e.g. storing **one million** Atari transitions in just **4 GB** of DRAM)).

2 Existing processing efficiency in PER

We conducted experiments to understand how sampling and vectorization affect non-tensor PER in both single-machine DQL and Ape-X DDQN using the Ray framework [10] across four Atari games: Beam Rider, Breakout, Qbert, and Space Invaders. Figure 1a illustrates that as the mini-batch size grows, more time is spent manipulating the replay buffer (**Buffer**), leaving fewer cycles for environment interaction (**Observe**) and neural network training (**Train**).



(a) Time usage breakdown in training



(b) Buffer operations broken down in one iteration

Although increasing the mini-batch size in DDQN training can enhance data processing throughput to a point (Figure 2a), it also reduces the rate of acquiring fresh transitions from the environment, ultimately curbing training efficiency. To pinpoint the main bottlenecks in PER, we decompose replay operations into three phases: *Store* (T_{st}) for adding new transitions, *Sample* (T_{sa}) for sampling from the buffer, and *Update* (T_u) for recalculating TD weights in the sum tree.

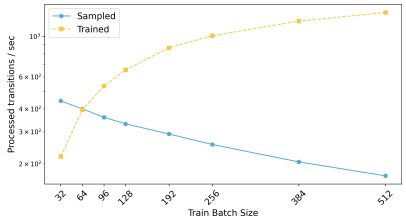
Figure 1b shows the time distribution in a single iteration across five phases: *Store*, *Sample*, *Update*, *Train* (time spent updating the network), and *Others* (environment interaction and additional framework overhead). It is evident that with larger mini-batch sizes, *Sample* and *Update* dominate runtime, while *Store* remains comparatively light.

A similar trend appears for Ape-X DDQN (Figure 2b). We limit the number of Buffer Actor to 1, and upgrade the training intensity while batch size rise, and analysis of processed frames across varying batch sizes similarly suggests that replay buffer overhead hinders linear throughput gains.

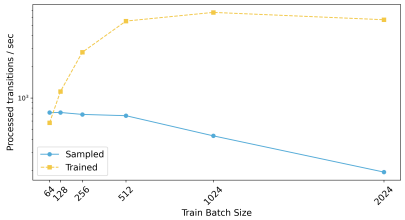
In summary, both single-machine and distributed PER implementations face an intensifying replay bottleneck as batch size increases. Additionally, as is well-known, larger replay buffers require substantially more DRAM—often reaching tens or hundreds of gigabytes—which further exacerbates resource constraints [5].

3 RASPBERry

Our proposed method, RAM Saving Prioritised Block Experience Replay (RASPBERry), offers a simple yet effective solution to challenges associated with replay buffers in DRL. RASPBERry aims to significantly reduce sampling overhead



(a) Processed transitions per seconds in DDQN



(b) Processed transitions per seconds in Ape-X DDQN

and DRAM capacity requirements in Experience Replay while maintaining training effectiveness, thus enabling DRL algorithms to be deployed on edge devices. This is achieved by compressing, storing, and updating blocks of transitions rather than handling each transition individually.

To enhance the conventional ER mechanism, RASPBERry introduces a dual-buffer framework:

- $\mathcal{D}$: The main replay buffer, similar to traditional ER, but stores transitions in blocks to reduce operational overhead.
- $\mathcal{D}_{\text{block}}$: An auxiliary storage buffer that temporarily holds new transitions. Once enough transitions accumulate to form a block, they are transferred to $\mathcal{D}$ as a single block.

Buffer $\mathcal{D}$ retains a flexible structure, akin to traditional PER (e.g., an ordered list combined with a sum tree for weights), allowing compatibility with dynamic buffer strategies.

RASPBERry preserves flexibility by minimally altering ER's overall organization. The auxiliary buffer $\mathcal{D}_{\text{block}}$ stores newly arrived transitions without changing the total count in $\mathcal{D}$, thus reducing the number of individual transitions the prioritization algorithm must manage. Moreover, transitions are stored in blocks as tensors $(B_{S_{t-1}}, B_{A_{t-1}}, B_{R_t}, B_{\gamma_t}, B_{S_t})$, each containing m transitions—similar to techniques used in Rainbow [11].

By treating each block in $\mathcal{D}$ as a single element during buffer operations, RASPBERry dramatically reduces the required sampling and update operations. For instance, with a batch size $M = 512$ and block size $m = 32$, the number of operations decreases from 512 to 16. Additionally, the sequential nature of transitions within a block allows for high compression rates. Using algorithms such as Deflate [4], 32 sequential Atari frames can be compressed into a binary format occupying only 0.2% to 5% of their original size. This makes it possible to store one million transitions in roughly 4 GB of DRAM, compared to 56.5 GB required by conventional PER—an ideal improvement for memory-constrained settings. The impact of block compression and decompression during sampling varies across environments. However, with parallel or asynchronous compression, the overhead introduced by compression is minimal.

Building on these principles, we implemented RASPBERry on top of Ray Rllib [10], as well as its distributed counterpart. Figures 3(a) and 3(b) illustrate a single training iteration in PER and RASPBERry, where the PER batch contains $M = 4$ transitions while $\mathcal{D}_{\text{block}}$ holds $m = 2$ transitions.

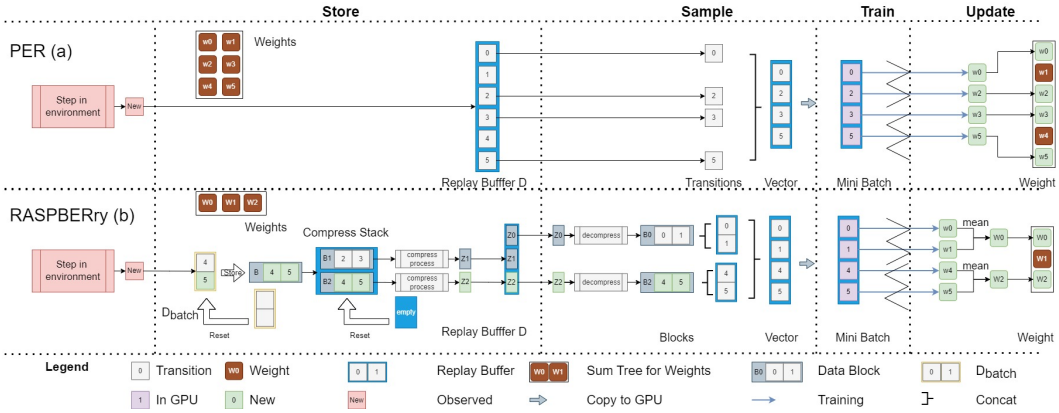


Figure 3: One training iteration in PER & PBER.

In RASPBERry, new transitions are first stored in $\mathcal{D}_{\text{batch}}$, then transferred to $\mathcal{D}$ as blocks, each block assigned a weight equal to the mean of its constituent transitions' priorities. This lowers buffer overhead from $t_{\text{ER}} = M \cdot t_{\text{sample}} + M \cdot t_{\text{update}}$ to $t_{\text{BER}} = \frac{M}{m} \cdot t_{\text{sample}} + \frac{M}{m} \cdot t_{\text{update}}$, improving time complexity from $O(\log N)$ to $O(\log(N/m))$.

Extending this approach to distributed frameworks, RASPBERry modifies the Replay and Learner components to handle blocks with asynchronous compression, maintaining high throughput while significantly reducing DRAM requirements. For example, In all Atari game, RASPBERry enabled an Ape-X agent with a 500k transition buffer to run on an laptop or edge devices with just 8 GB DRAM—a substantial reduction from the 70 GB typical of the original algorithm.

Overall, RASPBERry and its derivatives present robust solutions for optimizing replay buffers, achieving comparable training performance to traditional PER while drastically reducing computational and memory overhead, thus facilitating deployment on resource-constrained hardware. What is more, it is foreseeable that this optimization will also be effective for other ER methods such as HER [2], or PER in R2D2 [13].

4 Experiments

We evaluated RASPBERry and its distributed extensions using Ray-Project RLLib v2.8 [10], following the benchmark configurations from Ray-Project. Our experiments covered multiple Atari games as well as Box2D and MiniGrid [3] environments to assess the robustness and scalability of the proposed methods.

Figures 4 and 5 present smoothed learning curves for DDQN and Ape-X DDQN, respectively, contrasting our RASPBERry approach against standard PER/DPER. We observe that RASPBERry accelerates learning across most environments. This benefit stems from improved data throughput and lower buffer overhead, primarily reducing the **Sample** and **Update** times (Figure 6a)—particularly valuable when larger replay buffers are used, where traditional methods face computational and memory bottlenecks.

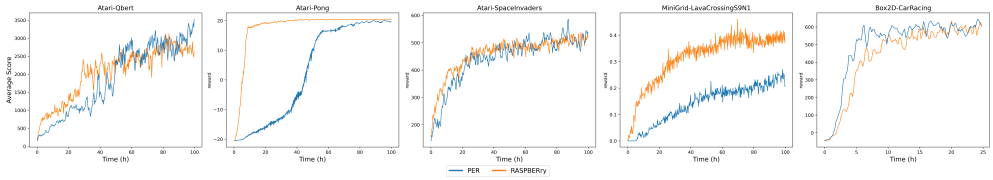


Figure 4: Smoothed learning curves of DDQN comparing PER and RASPBERry.

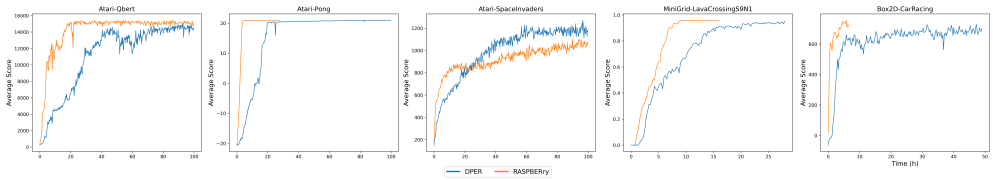
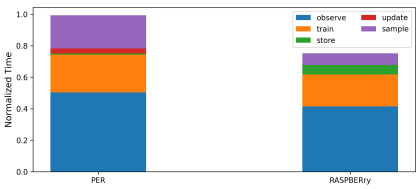
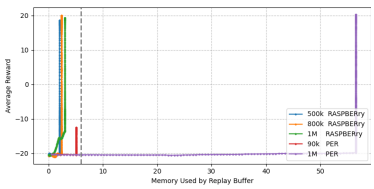


Figure 5: Smoothed learning curves of Ape-X DDQN comparing DPER and RASPBERry.

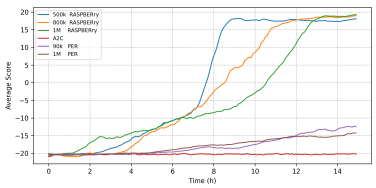
Furthermore, tests on simulated edge hardware (e.g., an NVIDIA Jetson TX2 with 8 GB of RAM) confirm that RASPBERry supports efficient DRL training under strict memory constraints. For example, when training Atari Pong (whose image frames compress well), RASPBERry stored 500k transitions in only 2.0 GB of DRAM. Figure 6b illustrates the memory usage for Pong. The gray dashed line indicates the maximum DRAM budget (8,GB) available for the replay buffer on our edge device. RASPBERry successfully remains below this limit even at higher buffer sizes, whereas PER would exceed it, rendering training infeasible on memory-constrained hardware. Meanwhile, Figure 6c shows the corresponding learning curve, illustrating that even under tight memory budgets, our approach retains effective performance.



(a) Operations time break down



(b) Memory boundary in Pong



(c) Learning curves of Pong on edge

5 Discussion and Future Work

Our experimental findings show that RASPBERry's primary advantages arise from reducing replay buffer overhead—particularly in *Sample* and *Update* operations. By compressing transitions into blocks, RASPBERry also alleviates

DRAM constraints, enabling large-scale or edge-device training that would be infeasible under standard PER. Notably, the compression approach is most effective for image-intensive tasks such as Atari Game; however, moderate gains are still achievable for other data types (e.g. MuJoCo states).

In scenarios such as Ape-X, we observe a marked increase in processed frames, indicating that the efficiency gains primarily stem from handling fewer replay elements overall. This would allows higher training intensity—a notable benefit in HPC settings. Future work may explore more advanced block-generation methods (e.g. integrating with Batch Prioritized Experience Replay) or extend RASPBERry to recurrent RL algorithms, such as R2D2. Additionally, while our experiments suggest block sizes near $\sqrt{M}$ can be beneficial, a more rigorous investigation of block-size selection and tuning remains open. Another potential direction is applying lossy compression techniques to see whether they can further reduce memory usage without severely harming training performance.

In conclusion, **RASPBERry** is a block-based replay approach that integrates seamlessly with single-machine and distributed RL frameworks. Our results demonstrate that RASPBERry reduces buffer overhead and memory use by up to 95%, while maintaining or improving learning performance. These benefits facilitate the use of large replay buffers and make DRL algorithms more accessible to resource-constrained devices. We hope this work catalyses further research on large-scale experience replay, block-level prioritization, weight calculation, and novel compression strategies for more efficient and broadly applicable DRL.

References

[1] David Andre, Nir Friedman, and Ronald Parr. Generalized Prioritized Sweeping. In M. Jordan, M. Kearns, and S. Solla, editors, *Advances in Neural Information Processing Systems*, volume 10. MIT Press, 1997.

[2] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight Experience Replay. In *Hindsight Experience Replay*. arXiv, February 2018. arXiv:1707.01495 [cs].

[3] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks, 2023. original-date: 2017-12-13T16:22:37Z.

[4] L. Peter Deutsch. DEFLATE Compressed Data Format Specification version 1.3. Request for Comments RFC 1951, Internet Engineering Task Force, May 1996. Num Pages: 17.

[5] William Fedus, Prajit Ramachandran, Rishabh Agarwal, Yoshua Bengio, Hugo Larochelle, Mark Rowland, and Will Dabney. Revisiting Fundamentals of Experience Replay, July 2020. arXiv:2007.06700 [cs, stat].

[6] Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado van Hasselt, and David Silver. Distributed Prioritized Experience Replay, March 2018. arXiv:1803.00933 [cs].

[7] Ang A. Li, Zongqing Lu, and Chenglin Miao. Revisiting Prioritized Experience Replay: A Value Perspective. *arXiv:2102.03261 [cs]*, February 2021. arXiv: 2102.03261.

[8] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3):293–321, May 1992.

[9] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015.

[10] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A Distributed Framework for Emerging AI Applications, September 2018. arXiv:1712.05889 [cs, stat].

[11] Jinwoo Park (Curt). Curt-Park/rainbow-is-all-you-need, November 2024. original-date: 2019-06-10T09:06:10Z.

[12] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[13] Jerome Revaud, Philippe Weinzaepfel, César De Souza, Noe Pion, Gabriela Csurka, Yohann Cabon, and Martin Humenberger. R2D2: Repeatable and Reliable Detector and Descriptor, June 2019. arXiv:1906.06195 [cs].

[14] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized Experience Replay, February 2016. arXiv:1511.05952.

[15] Yiqin Tan, Pihe Hu, Ling Pan, Jiatai Huang, and Longbo Huang. RLx2: Training a Sparse Deep Reinforcement Learning Model from Scratch, March 2023. arXiv:2205.15043 [cs].